LLM ENGINEERING

Bonus chapter 3

Bonus: GPU Kernels & the CuTe DSL



Bethgelab · Self-paced reading

University of Tübingen · Winter term 2026/27

Tübingen AI Center

Same softmax, same GPU:

why are five PyTorch ops 4.9× slower than one kernel?



Each eager operation is a round trip through HBM

- Eager chain: five kernels, one per operation
- Each kernel reads and writes HBM
- Fused kernel: one read, one write
- Few FLOPs per byte: bytes set the time

Orientation

When	Submission	Reflection questions
After Week 5, before Week 6	None (`scripts/kernels/` optional)	Three, discussed in the Week 6 tutorial

- Trace grid → cooperative thread array (CTA) → warp → thread
- Size residency and bytes in flight
- Map a thread-value layout to row offsets
- Explain the 4.9× from passes and bandwidth
- Trace fused softmax, connect it to FlashAttention

A kernel runs on thousands of threads

- **Kernel**: one function, run by every thread
- Launch: one call, a grid of threads
- PyTorch GPU operation: one or more kernels
- `torch.softmax`: one kernel per call

Five operations move four times the bytes of one

$$\text{traffic ratio} = \frac{8 \text{ passes (decomposed)}}{2 \text{ passes (fused)}} = 4\times \tag{1}$$

- Eager chain: 5 reads, 3 writes
- Fused: 1 read, 1 write
- Measured 4.9×: unequal bandwidths

UC3's H100, 32,768 × 4,096 bf16	Time	Kernels
Eager chain	1,248.0 µs	5
`torch.softmax`	466.8 µs	1
Fastest fused kernel (course `512x8`)	253.5 µs	1

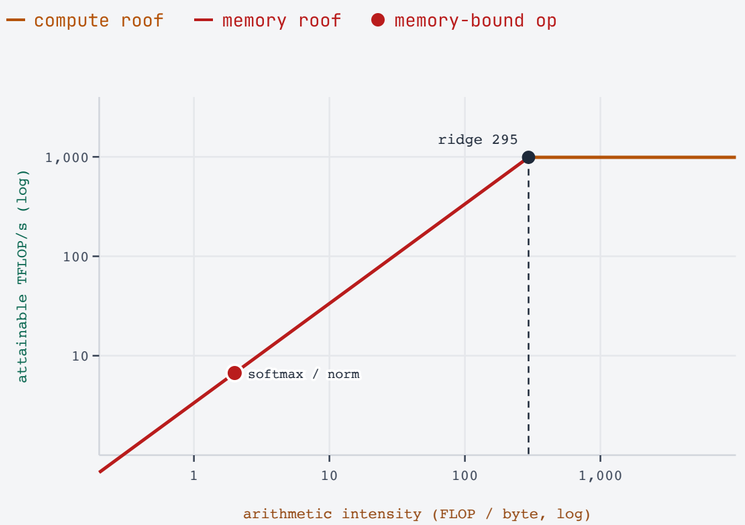
Softmax falls far below every ridge

- About 1.25 FLOP/byte: 5 operations, 4 bytes
- Drawn roof: bf16 Tensor Core peak, ridge 295
- fp32 roof: about 15× lower, ridge 20
- Chart marker: teaching value 2

H100 SXM, softmax

Where does a streamed reduction fall?

peak ≈ 989.5 TFLOP/s (dense bf16) · bandwidth ≈ 3.35 TB/s · ridge ≈ 295 FLOP/byte



Illustrative

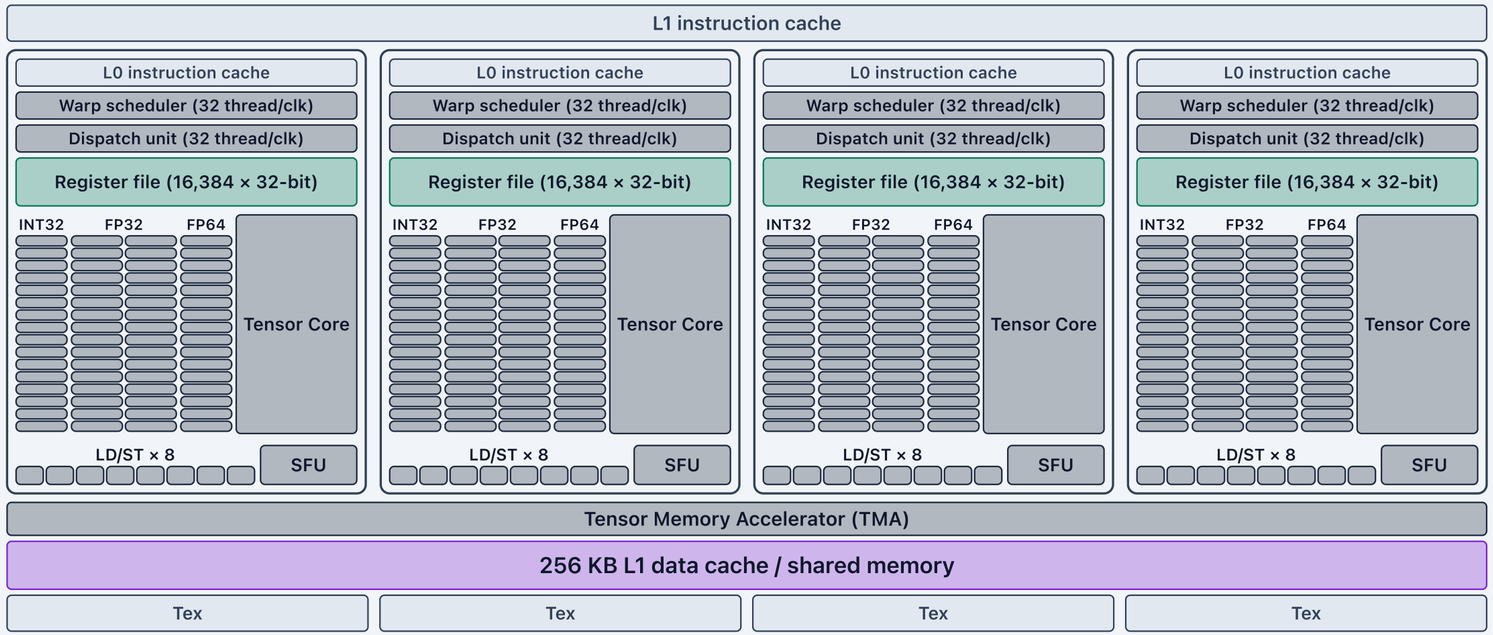
Library and compiler before a custom kernel

- Path: library → `torch.compile` → profiler → custom kernel
- Compiler: fuses pointwise and reduction chains
- Picks GEMM backends, one in the CuTe DSL (Patel et al., 2026)
- Custom kernel: only for a profiled boundary
- Done: correctness check, then fair benchmark

Each SM runs warps on four sub-partitions

GPU SM

One SM of 132 · 4 sub-partitions



Registers
256 KB per SM
bandwidth not given · a few ns

Shared memory
up to 228 KB per SM
31 TB/s peak, whole GPU · about 10–20 ns

L2 cache
50 MB
12 TB/s peak · about 150–200 ns

HBM3
80 GB
3.35 TB/s · about 400 ns

Execution units
warp schedulers, cores, Tensor Cores, SFUs, LD/ST units and the TMA

Hover over a part, or tab to it, to see what it does.

After Andersch et al., 2022, Figures 3 and 4

- Four sub-partitions, one warp scheduler each
- A warp stays on one sub-partition
- FP32 cores, special-function unit (SFU), Tensor Cores

Data lives in four memory tiers

Global memory (HBM) Registers Shared memory

Tier	H100 SXM size	Shared by	Rough latency
Registers	256 KB per SM (64K × 32 bit)	one thread	lowest
Shared memory	up to 228 KB per SM	one CTA	about 10–20 ns
L2 cache	50 MB	all SMs	about 150–200 ns
HBM	80 GB at 3.35 TB/s	all SMs	about 400 ns

- Week 4's SRAM: registers and shared memory
- Latencies: rounded, from the QuACK note (Guo et al., 2025)

Threads nest in warps, CTAs and a grid

- At most 1,024 threads per CTA
- Per SM: 64 warps, 2,048 threads, 32 CTAs

CONTAINMENT: LAUNCH → LANE

Grid — contains CTAs

CTA (thread block) — contains warps

Warp — contains 32 threads

Thread — one lane

Grid

EXECUTION SCOPE
one kernel launch

SCHEDULED ON
CTAs distributed across available SMs

STATE
kernel arguments and global memory (HBM) tensors

COOPERATION
separate launches or explicit grid synchronization

H100 SXM anchors (NVIDIA Hopper architecture blog, Nsight Compute guide, CUDA Programming Guide)

132 SMs

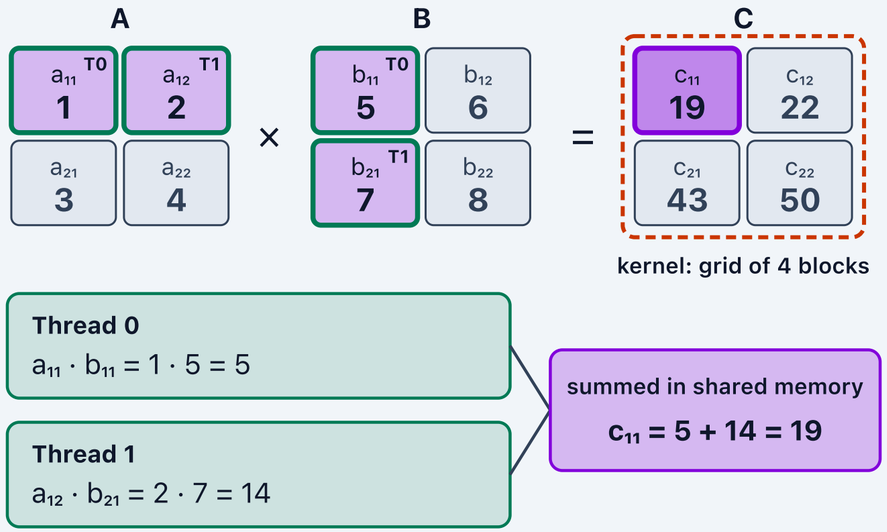
4 warp schedulers / SM

64 max active warps / SM

32 threads / warp

per SM: up to 32 CTAs, 64 warps, 2,048 threads

Kernel, block and thread split a 2×2 product



Level	In this example	Runs on
Kernel (grid)	all of C: 4 blocks	the GPU, its blocks spread over the SMs
Block (CTA)	c_{11}: row 1 of A, column 1 of B	one SM, start to finish
Warp	both threads, 30 of 32 lanes idle	one sub-partition
Thread	one product	a CUDA core (FP32 lane), one instruction at a time

- Kernel → GPU: all of C
- Block → SM: one entry c_{ij}
- Thread → CUDA core: one product
- Tensor Cores: one warp multiplies a whole tile

A warp issues one instruction for 32 threads

- **Warp**: 32 threads, one instruction stream
- Single instruction, multiple threads (SIMT)
- Shuffles: register exchange between lanes
- Since Volta: independent lane scheduling
- Explicit synchronization, no assumed lockstep

Each CTA stays on one SM

(2)
$$32\ \frac{\text{registers}}{\text{thread}} \times 1{,}024\ \frac{\text{threads}}{\text{CTA}} \times 2\ \text{CTAs} = 65{,}536 = 64\text{K registers per SM}$$

- CTA: threads sharing memory and barriers
- Grid: all CTAs of one launch
- Course kernel: 1,024 threads, 32 registers each
- Result: two CTAs per SM

Bytes in flight hide memory latency

$$\text{bytes in flight} = \text{bandwidth} \times \text{latency} \tag{3}$$

$$3.35\ \text{TB/s} \times 400\ \text{ns} \approx 1.3\ \text{MB} \approx 10\ \text{KB per SM} \tag{4}$$

- **Occupancy**: active warps ÷ maximum warps (NVIDIA, 2024a)
- Lever 1: more resident warps
- Lever 2: more bytes per load
- Occupancy: resident warps, not useful lanes

Neighboring lanes should read neighboring bytes

- 32-byte sectors: the unit of transfer (NVIDIA, 2026b)
- Cache line: four sectors, 128 bytes
- **Coalesced**: one warp, few sectors
- H100 loads: at most 16 bytes per thread (NVIDIA, 2026c)

The CuTe DSL compiles Python into a GPU kernel

- CuTe DSL: Python domain-specific language for CuTe (NVIDIA, 2024b)
- `@cute.kernel`: device code each thread runs
- `@cute.jit`: host code, or a helper a kernel calls
- `cute.compile`: compile once, reuse the binary
- Layout algebra: only in just-in-time (JIT) compiled functions

A layout is a shape and a stride

(5)

$$\text{offset}(c_0, c_1, \dots) = \sum_k c_k \, s_k \qquad (4,8) : (8,1) \colon \; (i,j) \mapsto 8i + j$$

- **Layout**: a shape plus a stride
- Written `shape:stride`
- Offset: each coordinate times its stride, summed

(4, 8):(8, 1)	i = 0	i = 1	i = 2	i = 3
j = 0	0	8	16	24
j = 1	1	9	17	25
j = 2	2	10	18	26
j = 3	3	11	19	27
j = 4	4	12	20	28
j = 5	5	13	21	29
j = 6	6	14	22	30
j = 7	7	15	23	31

Thread-value layouts are layouts too

- **Thread-value layout**: (thread t, value v) → offset
- Course kernel: `(1024, 4):(4, 1)`
- PyTorch's kernel: `(1024, 4):(1, 1024)` (PyTorch Contributors, 2026)
- Both cover each offset once, both coalesced

	`(1024, 4):(4, 1)`	`(1024, 4):(1, 1024)`
Offset	`4t + v`	`t + 1024v`
Thread 0	0, 1, 2, 3	0, 1024, 2048, 3072
Thread 1	4, 5, 6, 7	1, 1025, 2049, 3073
Thread 31	124, 125, 126, 127	31, 1055, 2079, 3103
Thread 1023	4092, 4093, 4094, 4095	1023, 2047, 3071, 4095
Loads per thread	one of 8 bytes	four of 2 bytes
One warp, one load	256 contiguous bytes	64 contiguous bytes

Which offsets does thread 5 own

under (1024, 4):(4, 1) and (1024, 4):(1, 1024)?

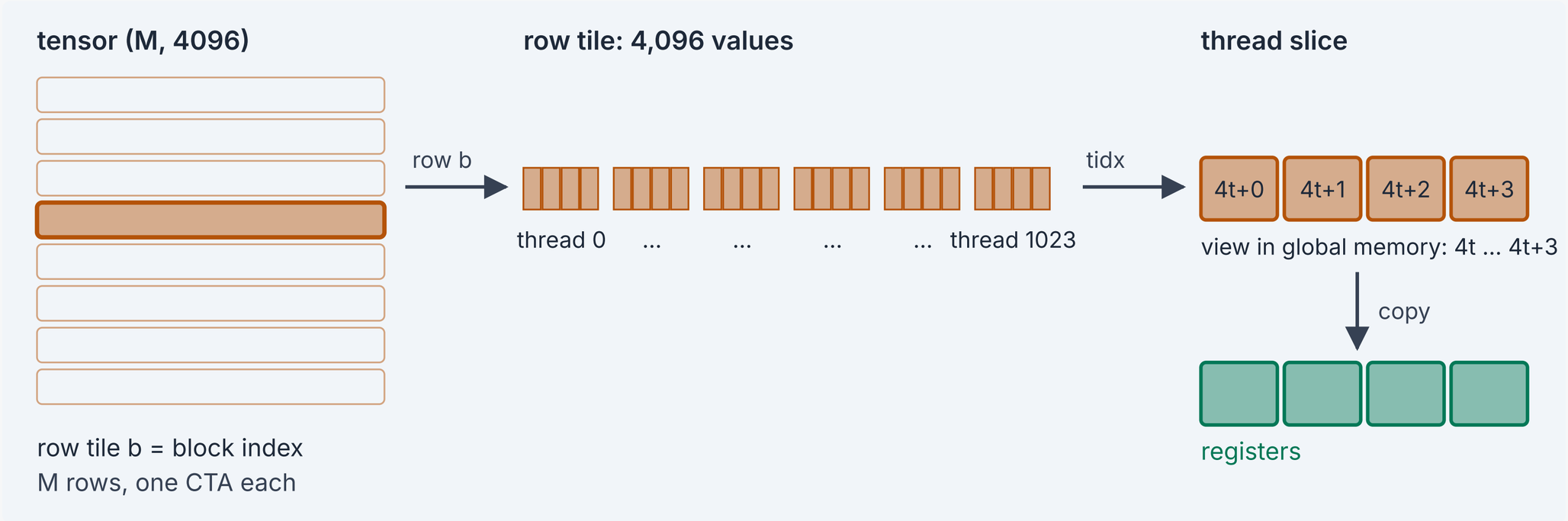


Thread 5 owns adjacent or scattered offsets

- `4t + v`: 20, 21, 22, 23
- `t + 1024v`: 5, 1029, 2053, 3077
- First: 8 adjacent bytes, one load
- Second: four 2-byte loads, 2,048 bytes apart

Tile by rows, then slice by thread

- `zipped_divide` by `(1, 4096)`: M row tiles
- `bidx`: this CTA's row
- `composition` with the layout, then `[tidx, None]`



Each row needs a maximum, a sum and a scale

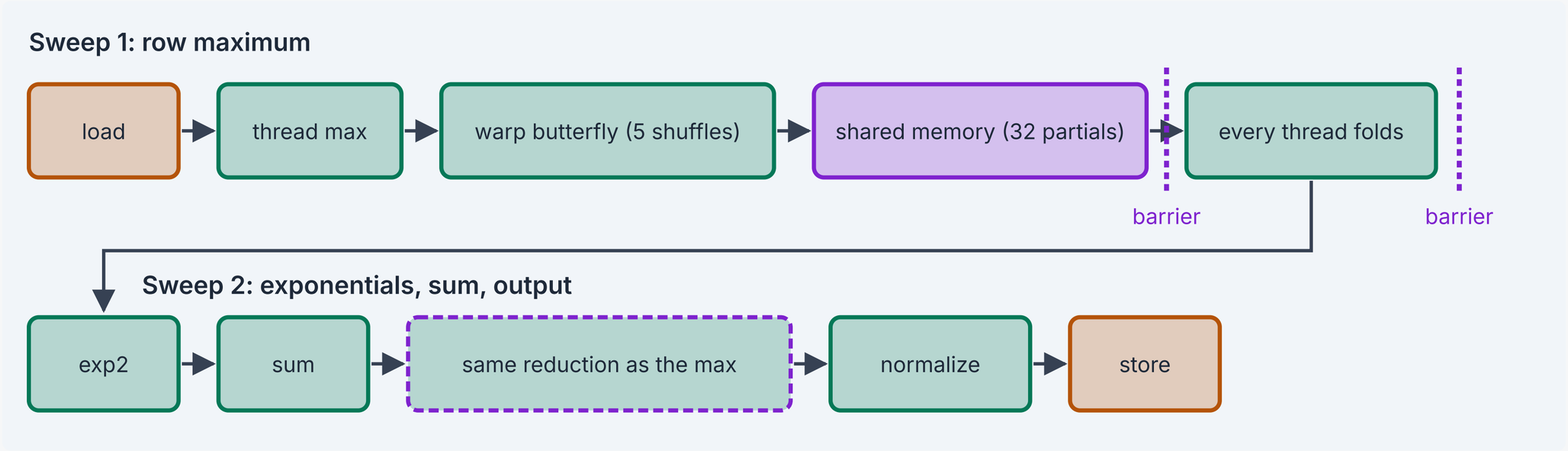
(6)
$$\mathrm{softmax}(x)_j = \frac{e^{x_j - m}}{l}, \qquad m = \max_k x_k, \qquad l = \sum_{k=1}^{N} e^{x_k - m}$$

Step	Eager chain	Fused kernel	Walkthrough
Launch, load the row	5 kernels	1 CTA per row, 1 read into registers	1, 2
Maximum m	1 read	reduction in three tiers	3
Exponentials $e^{x_j - m}$	2 reads, 2 writes	`exp2` on the SFU, in registers	3
Sum l	1 read	reduction in three tiers, in fp32	3
Divide, store	1 read, 1 write	1 reciprocal, 1 bf16 write	4

- Subtracting m keeps every exponential in $[0, 1]$
- Walkthroughs 1 to 4: the kernel's code, in order

One CTA per row, the row kept in registers

- Global memory: one read, one write
- Registers: the row, through both sweeps
- Shared memory: 32 warp partials only



Walkthrough 1: the host launches M CTAs

- Host function `softmax_host`: runs once per call
- Builds the layout, tiles the rows, launches the kernel

Global memory (HBM) Registers Shared memory

```python
@cute.jit
def softmax_host(mX: cute.Tensor, mO: cute.Tensor, stream: cuda.CUstream):
    # Thread-value layout: (thread t, value v) -> offset within the row.
    tv_layout = cute.make_layout((threads, vec), stride=tv_stride)
    # Cut the (M, 4096) tensors into M row tiles of shape (1, 4096).
    gX = cute.zipped_divide(mX, (1, SUPPORTED_N))
    gO = cute.zipped_divide(mO, (1, SUPPORTED_N))
    # One CTA per row tile.
    softmax_kernel(gX, gO, tv_layout).launch(
        grid=[cute.size(gX, mode=[1]), 1, 1],
        block=[threads, 1, 1],
        stream=stream,
    )
```

Walkthrough 2: each thread loads four values

- Kernel `softmax_kernel` begins: every thread runs it
- Each thread loads its four values once

Global memory (HBM) Registers Shared memory

```python
@cute.kernel
def softmax_kernel(gX: cute.Tensor, gO: cute.Tensor, tv_layout: cute.Layout):
    # This thread's index in its CTA, and the CTA's index in the grid.
    tidx, _, _ = cute.arch.thread_idx()
    bidx, _, _ = cute.arch.block_idx()
    # One fp32 slot per warp in shared memory, for the warp partials.
    smem = cutlass.memory.SmemAllocator()
    reduction_buffer = smem.allocate_tensor(
        Float32, cute.make_layout(num_warps), byte_alignment=4
    )
    # This CTA's row (block index bidx), then this thread's values in it.
    tXgX = cute.composition(gX[((None, None), bidx)], tv_layout)[tidx, None]
    tXgO = cute.composition(gO[((None, None), bidx)], tv_layout)[tidx, None]
    # Copy the bf16 values from global memory into registers, widen to fp32.
    frgX = cute.make_rmem_tensor(cute.size(tXgX), gX.element_type)
    cute.autovec_copy(tXgX, frgX)
    x = frgX.load().to(Float32)
```

Walkthrough 3: two reductions in three tiers

- Kernel continued: maximum, exponentials, sum
- Helper `row_reduce`, called twice: three-tier reduction

Global memory (HBM) Registers Shared memory

```
    row_max = row_reduce(x, cute.ReductionOp.MAX, reduction_buffer, -Float32.inf)
    # e^(x - max) computed as 2^((x - max) * log2 e) on the special-function unit.
    exp_x = cute.math.exp2(x * log2_e - row_max * log2_e, fastmath=True)
    row_sum = row_reduce(exp_x, cute.ReductionOp.ADD, reduction_buffer, Float32(0.0))


@cute.jit
def row_reduce(x: cute.TensorSSA, op: cutlass.Constexpr, reduction_buffer: cute.Tensor,
               init_val: cute.Numeric) -> Float32:
    fold = {cute.ReductionOp.MAX: cute.arch.fmax, cute.ReductionOp.ADD: operator.add}[op]
    # Tier 1: combine this thread's own values.
    val = x.reduce(op, init_val=init_val, reduction_profile=0)
    # Tier 2: a butterfly over the warp's 32 lanes, five register shuffles.
    for i in cutlass.range_constexpr(5):
        val = fold(val, cute.arch.shuffle_sync_bfly(val, offset=1 << i))
    # Tier 3: lane 0 of each warp writes its partial to shared memory.
    # Read both indices before the branch: warp_idx() is a warp-wide
    # shuffle, which deadlocks if only lane 0 executes it.
    lane = cute.arch.lane_idx()
    warp = cute.arch.warp_idx()
    if lane == 0:
        reduction_buffer[warp] = val
    cute.arch.barrier()
    # Every thread folds all partials, so every thread holds the row result.
    acc = reduction_buffer[0]
    for w in cutlass.range_constexpr(1, num_warps):
        acc = fold(acc, reduction_buffer[w])
    # Before anyone writes the buffer again, every thread must finish reading it.
    cute.arch.barrier()
    return acc
```

Walkthrough 4: normalize in fp32, store bf16 once

- Kernel ends: scale, narrow to bf16, store once

(7)

$$e^{x-m} = 2^{(x-m)\log_2 e}$$

Global memory (HBM) Registers Shared memory

```
# Normalize in fp32, narrow to bf16, write the row back once.
y = exp_x * cute.arch.rcp_approx(row_sum)
frgO = cute.make_rmem_tensor(cute.size(tXgO), gO.element_type)
frgO.store(y.to(gO.element_type))
cute.autovec_copy(frgO, tXgO)
```

fp32 accumulation protects the normalizer

$$\underbrace{1+1+\cdots+1}_{4{,}096\ \text{terms}} = \begin{cases} 256 & \text{summed in order in bf16} \\ 4{,}096 & \text{in fp32} \end{cases} \tag{8}$$

- bf16, fp32: same 8-bit exponent
- After max subtraction: exponentials in [0, 1]
- Risk: rounding in long sums
- Constructed example: bf16 running sum stalls at 256

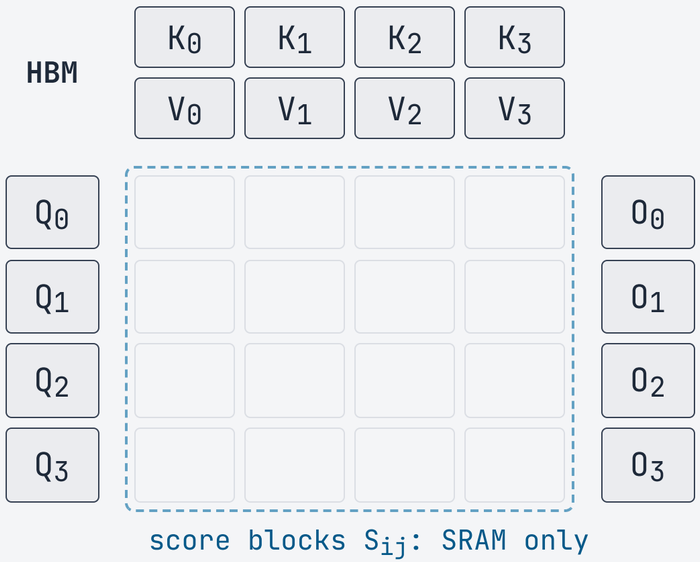
On UC3's H100, 512x8 and QuACK reach copy speed

Implementation	Median µs	GB/s	% of copy	% of peak	Spread	Kernels
Copy (`out.copy_(x)`)	251.7	2,132.9	100.0%	89.0%	0.0%	0 (1 copy)
Eager chain	1,248.0	430.2	20.2%	18.0%	0.5%	5
`torch.softmax`	466.8	1,150.2	53.9%	48.0%	24.3%	1
`torch.compile`	359.3	1,494.1	70.1%	62.4%	6.9%	1
Course `1024x4`	369.4	1,453.5	68.1%	60.7%	2.1%	1
Course `strided`	377.6	1,421.8	66.7%	59.3%	5.6%	1
Course `512x8`	253.5	2,118.0	99.3%	88.4%	0.2%	1
QuACK	254.5	2,109.5	98.9%	88.0%	0.1%	1

- Compiler: five kernels into one
- Layout: 2% gap, within the 5.6% spread
- 512×8: 16-byte loads, half the warps

Week 4 recap: K/V blocks pass one Q block

ILLUSTRATIVE | ◀ Prev | ▶ Play | Next ▶ | Reset | start



SRAM (on chip)

```
for each Q block i:
  load Qi; mi = -∞, ℓi = 0, Oi = 0
  for each K/V block j:
    load Kj, Vj
    S = Qi Kj^T in SRAM
    update mi, ℓi, Oi online softmax
  Oi = Oi / ℓi
  write Oi and Li = mi + log ℓi to HBM
```

score blocks in HBM	**0**
O blocks written	**0 / 4**
K/V block loads	**0**

in SRAM now | × discarded | written to HBM

Attention streams its row and carries m, l, o

(9) per K/V block

$$
\begin{aligned}
m_{\text{new}} &= \max\left(m_{\text{old}},\ \max_j s_j\right) \\
l_{\text{new}} &= e^{m_{\text{old}} - m_{\text{new}}}\, l_{\text{old}} + \sum_j e^{s_j - m_{\text{new}}} \\
o_{\text{new}} &= e^{m_{\text{old}} - m_{\text{new}}}\, o_{\text{old}} + \sum_j e^{s_j - m_{\text{new}}}\, v_j
\end{aligned}
$$

(10)

$$
\text{output} = o/l \quad \text{(once, after the last block)}
$$

- Row too long to keep: stream K/V blocks
- Carried state: running max, sum, output
- Rescale `l` and `o` when `m` grows

What are m, l, o after each tile?

Tile 1: scores (1, 2), values (1, 0)
Tile 2: scores (3, 0), values (2, 1)



Two rescaled tiles match one softmax

After	`m`	`l`	`o`
Tile 1	2	1.368	0.368
Tile 2	3	1.553	2.185

- Rescale factor: `e^(2 - 3)` ≈ 0.368
- `o / l` ≈ 1.407: same as attention over all four

FlashAttention adds Tensor Cores and pipelining

- Q, K, V tiles sized for shared memory
- Tensor Cores: matrix multiply-accumulate (MMA) instructions
- Hopper: four warps, one warpgroup, one MMA
- Tensor Memory Accelerator (TMA): asynchronous tile copies
- Warp specialization: producer and consumer warps

FlashAttention-4 rebalances the work for Blackwell

Per SM, per clock	Hopper	Blackwell B200
bf16 MMA operations	4,096	8,192
Exponentials	16	16
Shared memory bytes	128	128

- Matrix units doubled, special-function unit unchanged (Zadouri et al., 2026)
- 10–25% of exponentials on fused multiply-add units
- Rescale only for max increases above 8

Read production kernels by their roles

- Four roles: layout, partition, reduction or MMA, pipeline
- QuACK: `_threads_per_row`, `row_reduce`, clusters (Dao-AILab, 2026)
- FlashAttention-4: tiles, MMA, rescaling, producer warps (Zadouri et al., 2026)

So why are five PyTorch ops 4.9× slower than one kernel?



A quarter of the bytes, moved 1.23 times faster

$$\frac{8\ \text{passes}}{2\ \text{passes}} \times \frac{2{,}118.0\ \text{GB/s}}{1{,}721\ \text{GB/s}} \approx 4.9$$

- Eager: 8 passes through HBM, fused: 2
- Eager chain averages 81% of copy speed
- Fused: the row stays in registers
- Copy speed needs enough bytes in flight
- `512x8` and QuACK: 16-byte loads reach it

Reflection questions

No submission. Bring short answers to the Week 6 tutorial.

1. An eager RMSNorm in bf16 squares `x`, takes each row's mean, applies `rsqrt`, multiplies `x` by the result and then by the weight `g`, one kernel per step. Count its full-tensor passes against a fused kernel and state the ceiling with its assumptions. The course's `RMSNorm` upcasts `x` to fp32 before these steps and casts the result back. How does that change the count and the ceiling?
2. A row-wise kernel such as softmax meets rows of 128 bf16 values in one model and rows of 262,144 in another. Would you give each row a warp, a CTA or a cluster of CTAs? Argue from useful lanes, bytes in flight, registers and shared memory.
3. In bf16, a running sum of 4,096 ones stops at 256. Name two other places in one training step of the course model where a long sum in bf16 would lose precision in the same way, and say how the course code or PyTorch keeps each sum accurate.